BLOCK 34

Writing Files and Building Persistent Output

Day 5 • Block 34 • 30 minutes teaching + 30 minutes exercises

From screen-only programs to repeatable file output

Key shift: printed output disappears; written output can be inspected, shared, compared, and rerun.

BLOCK 34

What Students Should Be Able to Do

The goal is not memorizing file syntax; the goal is safe persistence.

Choose the right file mode	read, write, or append
Write text files safely	including newline characters
Create output folders	without manual setup every time
Separate raw and output data	so source files stay unchanged
Build a small pipeline	read → process → write

Mental model

A program can create durable artifacts — files that remain after the Python process ends.

BLOCK 34

Reading Versus Writing

Opening a file means choosing what kind of relationship the program has with it.

Read

```
with open("data/raw/ids.txt",
"r",
          encoding="utf-8") as
file:
    contents = file.read()
```

The program consumes existing file contents.

Write

```
with
open("data/output/summary.txt",
"w",
          encoding="utf-8") as
file:
    file.write("Processing
complete")
```

The program creates or replaces file contents.

BLOCK 34

File Modes Are Intentional

The mode tells Python what kind of operation is allowed.

Mode	Meaning	Beginner mental model
`"r"`	**Read**	Use existing contents
`"w"`	**Write**	Create or replace contents
`"a"`	**Append**	Add to the end
`"x"`	**Create**	Fail if file already exists

In this course, students mostly use "r", "w", and "a".

BLOCK 34

The Dangerous Power of "w"

Write mode is useful because it is destructive. Treat it with respect.

Before program runs

```
summary.txt

Old results
Records: 15
Errors: 2
```

After program runs

```
summary.txt

Processing complete
```

```
open(..., "w")
```

Important: "w" does not add a new version. It replaces the file contents.

BLOCK 34

Writing a Text File

The same context-manager pattern from reading also protects writing.

```
with open(
    "data/output/summary.txt",
    "w",
    encoding="utf-8"
) as file:

    file.write("Processing complete")
```

What happens?

If the file does not exist, Python creates it.

If the file already exists, Python replaces it.

BLOCK 34

Writing Multiple Lines

A file does not automatically move to the next line after write().

```
with open("data/output/summary.txt", "w",
          encoding="utf-8") as file:

    file.write("Processing Summary\n")
    file.write("Records: 100\n")
    file.write("Errors: 3\n")
```

The newline character

`\n` means “start the next text on a new line.”

Without it, values may run together.

BLOCK 34

Common Mistake: Missing Newlines

Repeated writes do exactly what you ask — no more, no less.

Missing newline

```
for record_id in
valid_record_ids:
    file.write(record_id)
```

```
EQ-1001EQ-1002EQ-1004
```

Correct newline

```
for record_id in
valid_record_ids:
    file.write(record_id + "\n")
```

```
EQ-1001
EQ-1002
EQ-1004
```

Each line in the output file usually needs its own "\n".

BLOCK 34

Appending Adds to the End

Append mode is useful for logs and history-style output.

```
with open("data/output/log.txt", "a",
          encoding="utf-8") as file:

    file.write("Processing started\n")
```

Run program twice

log.txt

```
Processing started
Processing started
```

Nothing is replaced; a new line is added each time.

BLOCK 34

Separate Raw Data from Output Data

This is a professional habit, not just a folder preference.

Avoid

```
Open source file
Change it manually
Save over original
```

Hard to recover. Hard to audit.
Hard to rerun.

Prefer

```
data/
    raw/
        source_data.csv

    output/
        cleaned_data.csv
```

Raw input remains unchanged;
output can be regenerated.

BLOCK 34

Input → Process → Output

The source file should not be the same file as the product.

1. Read
load existing values

2. Normalize
strip, uppercase

3. Validate
accept or reject

4. Write
persist new files

BLOCK 34

Create Output Directories in Code

Do not require a human to manually create every folder first.

```
from pathlib import Path

output_directory = Path("data/output")

output_directory.mkdir(
    parents=True,
    exist_ok=True
)
```

Two important options

`parents=True`

create missing parent folders

`exist_ok=True`

do not fail if folder already exists

BLOCK 34

Construct Paths with /

Path objects let you build file locations without string gymnastics.

```
from pathlib import Path

output_directory = Path("data/output")

output_file = (
    output_directory
    / "summary.txt"
)
```

Why this matters

You are composing a path intentionally instead of gluing together strings by hand.

BLOCK 34

Mini-Demo: Write the Valid IDs

Block 34 builds directly on the reading and validation logic from Block 33.

```
valid_record_ids = [
    "EQ-1001",
    "EQ-1002",
    "EQ-1004",
]

output_file = Path("data/output/valid_ids.txt")

with open(output_file, "w", encoding="utf-8") as file:

    for record_id in valid_record_ids:
        file.write(record_id + "\n")
```

Output file

valid_ids.txt

```
EQ-1001
EQ-1002
EQ-1004
```

BLOCK 34

Guided Exercise: 30 Minutes

Students practice writing files before combining reading and writing.

1. Write a summary file
2. Write valid IDs one per line
3. Write rejected values
4. Append to a log and run twice

Timer: 30 minutes total — work in small steps and inspect each output file.

Exercise 1: Write a Summary File

Create a text file with three lines of output.

Task

```
output/processing_summary.t
xt

Records processed: 20
Records valid: 18
Records rejected: 2
```

```
from pathlib import Path

output_file =
Path("output/processing_summary.txt")

with open(output_file, "w", encoding="utf-8")
as file:
    # write three lines here
```

Timebox: 6 minutes

BLOCK 34

Exercise 2: Write IDs One Per Line

Given a list, produce a readable output file.

```
valid_record_ids = [
    "EQ-1001",
    "EQ-1002",
    "EQ-1004",
]
```

Write these values to valid_ids.txt with one ID on each line.

```
output_file = Path("output/valid_ids.txt")

with open(output_file, "w", encoding="utf-8")
as file:

    for record_id in valid_record_ids:
        # write record_id and a newline
```

Timebox: 7 minutes

Exercise 3: Create a Rejected Records File

Output files are not only for successful values.

```
invalid_values = [
    "BAD-1003",
    "INVALID",
]
```

Write these values to rejected_records.txt. Use one value per line.

```
output_file =
Path("output/rejected_records.txt")

with open(output_file, "w", encoding="utf-8")
as file:

    for value in invalid_values:
        file.write(value + "\n")
```

Timebox: 7 minutes

Exercise 4: Append to a Log File

Run it twice and inspect how the file changes.

```
log_file = Path("output/log.txt")

with open(log_file, "a", encoding="utf-8") as
file:
    file.write("Processing started\n")
    file.write("Processing finished\n")
```

Observe

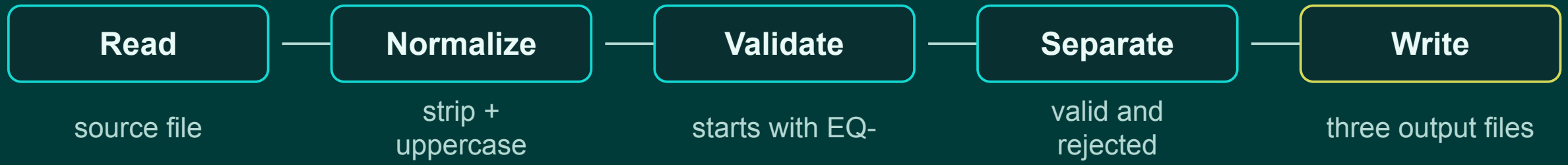
Does the second run replace the file or add more lines?

Timebox: 6 minutes

BLOCK 34

Applied Challenge: Text File Processor

Combine Block 33 reading with Block 34 writing.



Rule: the source file must not be changed.

BLOCK 34

Challenge File Layout

Students should see exactly where input and output belong.

Input

```
data/
    raw/
        record_ids.txt
```

Outputs

```
data/
    output/
        valid_ids.txt
        rejected_ids.txt
        summary.txt
```

Create missing output folders in code.

BLOCK 34

Challenge Processing Rules

Make the program explicit and inspectable.

1. Read every line
2. Strip whitespace
3. Ignore blank lines
4. Convert to uppercase
5. Accept only values beginning with EQ-
6. Store invalid values separately
7. Write valid, rejected, and summary files

BLOCK 34

Challenge Scaffold: Read and Separate

Students fill in the validation condition and list operations.

```
from pathlib import Path

input_file = Path("data/raw/record_ids.txt")

valid_ids = []
rejected_ids = []

with open(input_file, encoding="utf-8") as file:

    for line in file:
        record_id = line.strip().upper()

        if record_id == "":
            continue

        # separate valid and rejected IDs
```

Goal

Create two Python lists that can be written to files on the next slide.

BLOCK 34

Challenge Scaffold: Write the Products

A pipeline is not complete until results are persisted.

```
output_directory = Path("data/output")
output_directory.mkdir(parents=True, exist_ok=True)

valid_file = output_directory / "valid_ids.txt"
rejected_file = output_directory / "rejected_ids.txt"
summary_file = output_directory / "summary.txt"

# write valid IDs
# write rejected IDs
# write summary counts
```

Expected summary

```
Valid IDs: 4
Invalid IDs: 1
```

Counts should come from the lists.

Block 34 Takeaways

Students now have the write side of the file-processing pipeline.

"w" replaces	use carefully
"a" appends	good for logs
"\n" matters	files need line breaks
raw ≠ output	preserve the source
Path + mkdir	make output repeatable

Next: CSV files turn rows and columns into Python records.